

Лабораторная работа 9

Вывод шестнадцатеричного числа, содержащегося в одном слове¹. Цель работы

Цель лабораторной работы - разработать и отладить две различные процедуры вывода на экран шестнадцатеричного числа, содержащегося в одном машинном слове, то есть в двух байтах.

Первая процедура использует алгоритм последовательного деления числа на основание системы счисления. Вторая процедура выводит слово через два вызова ранее разработанной процедуры `Write_byte_hex`.

2. Постановка задачи

- написать процедуру вывода 16-битного числа в шестнадцатеричном виде по алгоритму деления;
- написать процедуру `Write_word_hex`, которая выводит старший и младший байты двумя вызовами `Write_byte_hex`;
- разместить тестирующую программу и процедуры в разных исходных файлах;
- протестировать работу процедур на граничных и типовых значениях;
- подготовить отчет со скриншотами выполнения программы.

3. Структура исходных файлов

Файл	Назначение	Основные элементы
VIDEO_IO.ASM	Файл с процедурами вывода.	<code>Write_digit_hex</code> , <code>Write_byte_hex</code> , <code>Write_word_hex_div</code> , <code>Write_word_hex</code> , <code>Write_crlf</code>
TEST9.ASM	Тестирующая программа.	Подключение процедур через <code>EXTRN</code> , набор тестовых значений, вывод результатов

4. Описание алгоритмов

4.1. Первая процедура `Write_word_hex_div`

Процедура получает число в регистре `AX`. Далее число последовательно делится на 16. Остатки от деления сохраняются в стеке. После завершения деления остатки извлекаются из стека и выводятся в обратном порядке. Такой подход аналогичен стандартному алгоритму вывода десятичных чисел через деление на основание системы счисления.

Особенность процедуры: ведущие нули не выводятся. Например, значение `000Fh` выводится как `F`.

4.2. Вторая процедура `Write_word_hex`

Процедура получает слово в `AX` и выводит его как ровно четыре шестнадцатеричных символа. Сначала выводится старший байт `AH`, затем младший байт `AL`. Для каждого байта вызывается процедура `Write_byte_hex`, разработанная в предыдущей лабораторной работе.

Особенность процедуры: адреса и слова всегда отображаются в виде четырех символов. Например, 000Fh выводится как 000F.

5. Исходный код программы

5.1. Файл VIDEO_IO.ASM

```
; =====
; Файл: VIDEO_IO.ASM
; Назначение: процедуры вывода шестнадцатеричных чисел.
; Автор отчёта: Элизбарян Эмин
; =====

.MODEL SMALL
.CODE

PUBLIC Write_digit_hex
PUBLIC Write_byte_hex
PUBLIC Write_word_hex_div
PUBLIC Write_word_hex
PUBLIC Write_crlf

; -----
; Write_digit_hex
; Ввод: AL - число от 0 до 0Fh.
; Вывод: на экран выводится один шестнадцатеричный символ.
; Использует: DOS int 21h, функция 02h.
; -----
Write_digit_hex PROC NEAR
    push ax                ; сохраняем AX
    push dx                ; сохраняем DX

    and al, 0Fh            ; оставляем только младшие 4 бита
    cmp al, 09h            ; цифра 0..9?
    jbe short digit_is_num ; если да, переход
    add al, 07h            ; для A..F корректируем код

digit_is_num:
    add al, '0'            ; перевод значения в ASCII-код
    mov dl, al             ; символ помещаем в DL
    mov ah, 02h            ; функция вывода символа
    int 21h

    pop dx
    pop ax
    ret
Write_digit_hex ENDP

; -----
; Write_byte_hex
; Ввод: AL - байт, который нужно вывести.
; Вывод: две шестнадцатеричные цифры.
; Алгоритм: сначала выводится старшая тетрада, затем младшая.
; -----
Write_byte_hex PROC NEAR
    push ax
    push cx

    mov ah, al             ; сохраняем исходный байт в AH
    mov cl, 04h            ; количество сдвигов
    shr al, cl             ; старшая тетрада становится младшей
    call Write_digit_hex   ; вывод первой цифры

    mov al, ah             ; восстанавливаем исходный байт
    and al, 0Fh            ; выделяем младшую тетраду
    call Write_digit_hex   ; вывод второй цифры

    pop cx
    pop ax
    ret
Write_byte_hex ENDP
```

```

; -----
; Write_word_hex_div
; Ввод: AX - слово, которое нужно вывести.
; Вывод: шестнадцатеричное число без ведущих нулей.
; Алгоритм аналогичен алгоритму вывода десятичного числа:
; деление на основание системы счисления, запись остатков в стек,
; затем вывод остатков в обратном порядке.
; -----
Write_word_hex_div PROC NEAR
    push ax
    push bx
    push cx
    push dx

    xor cx, cx                ; счётчик полученных цифр
    cmp ax, 0000h            ; отдельная проверка нуля
    jne short div_loop

    mov al, 00h              ; если AX=0, выводим одну цифру 0
    call Write_digit_hex
    jmp short div_done

div_loop:
    xor dx, dx                ; перед div очищаем старшую часть
    mov bx, 0010h            ; основание шестнадцатеричной системы
    div bx                    ; DX:AX / 16, остаток в DX
    push dx                  ; сохраняем остаток в стеке
    inc cx                    ; увеличиваем количество цифр
    cmp ax, 0000h            ; пока частное не равно нулю
    jne short div_loop

print_loop:
    pop ax                    ; извлекаем очередную цифру
    call Write_digit_hex      ; выводим её как HEX-символ
    loop print_loop

div_done:
    pop dx
    pop cx
    pop bx
    pop ax
    ret

Write_word_hex_div ENDP

; -----
; Write_word_hex
; Ввод: AX - слово, которое нужно вывести.
; Вывод: ровно четыре шестнадцатеричные цифры.
; Алгоритм: два вызова Write_byte_hex:
; 1) вывод старшего байта AH;
; 2) вывод младшего байта AL.
; -----
Write_word_hex PROC NEAR
    push ax                    ; сохраняем исходное слово
    mov al, ah                 ; переносим старший байт в AL
    call Write_byte_hex        ; выводим старший байт
    pop ax                     ; восстанавливаем исходное слово
    call Write_byte_hex        ; выводим младший байт
    ret

Write_word_hex ENDP

; -----
; Write_crlf
; Выполняет перевод строки: CR + LF.
; -----
Write_crlf PROC NEAR
    push ax
    push dx

    mov ah, 02h
    mov dl, 0Dh                ; возврат каретки
    int 21h
    mov dl, 0Ah                ; перевод строки
    int 21h

    pop dx
    pop ax

```

```
        ret
Write_crlf ENDP

END
```

5.2. Файл TEST9.ASM

```
; =====
; Файл: TEST9.ASM
; Назначение: тестирующая программа для процедур из VIDEO_IO.ASM.
; Автор отчёта: Элизбарян Эмин
; =====

.MODEL SMALL
.STACK 100h

.DATA
msg_title db 'LAB 9. WORD HEX OUTPUT TEST',0Dh,0Ah,'$'
msg_value db 'Input word = $'
msg_alg1  db 'Method 1   = $'
msg_alg2  db 'Method 2   = $'

.CODE
EXTRN Write_word_hex_div:NEAR
EXTRN Write_word_hex:NEAR
EXTRN Write_crlf:NEAR

start:
    mov ax, @data
    mov ds, ax

    lea dx, msg_title
    call Print_string
    call Write_crlf

    mov ax, 0000h
    call Test_value
    mov ax, 0009h
    call Test_value
    mov ax, 000Ah
    call Test_value
    mov ax, 000Fh
    call Test_value
    mov ax, 0010h
    call Test_value
    mov ax, 00FFh
    call Test_value
    mov ax, 0F0Fh
    call Test_value
    mov ax, 1234h
    call Test_value
    mov ax, 0FFFFh
    call Test_value

    mov ax, 4C00h
    int 21h

; -----
; Print_string
; Ввод: DS:DX - адрес строки, оканчивающейся символом '$'.
; -----
Print_string PROC NEAR
    mov ah, 09h
    int 21h
    ret
Print_string ENDP

; -----
; Test_value
; Ввод: AX - проверяемое слово.
; Действие: выводит слово двумя способами.
; -----
Test_value PROC NEAR
    push ax
    lea dx, msg_value
    call Print_string
    pop ax
    push ax
    call Write_word_hex      ; контрольный вывод в 4 цифры
    call Write_crlf

    lea dx, msg_alg1
```

```

call Print_string
pop ax
push ax
call Write_word_hex_div ; первая процедура
call Write_crlf

lea dx, msg_alg2
call Print_string
pop ax
call Write_word_hex ; вторая процедура
call Write_crlf
call Write_crlf
ret
Test_value ENDP

END start

```

6. Тестовые значения

Для проверки были выбраны граничные и типовые значения 16-битного слова.

Значение AX	Процедура 1	Процедура 2	Пояснение
0000	0	0000	нулевое значение
0009	9	0009	последняя десятичная цифра
000A	A	000A	первая буква A
000F	F	000F	максимум одной тетрады
0010	10	0010	переход к двум тетрадам
00FF	FF	00FF	максимум одного байта
0F0F	F0F	0F0F	смешанное значение
1234	1234	1234	обычное тестовое слово
FFFF	FFFF	FFFF	максимальное слово

7. Скриншоты выполнения

Ниже приведены экранные изображения редактирования, сборки, запуска и отладки программы.

```

001 ; =====
002 ; Файл: VIDEO_IO.ASM
003 ; Назначение: процедуры вывода шестнадцатеричных чисел.
004 ; Автор отчёта: Элизбарян Эмин
005 ; =====
006
007 .MODEL SMALL
008 .CODE
009
010 PUBLIC Write_digit_hex
011 PUBLIC Write_byte_hex
012 PUBLIC Write_word_hex_div
013 PUBLIC Write_word_hex
014 PUBLIC Write_crlf
015
016 ; -----
017 ; Write_digit_hex
018 ; Ввод: AL - число от 0 до 0Fh.
019 ; Вывод: на экран выводится один шестнадцатеричный символ.
020 ; Использует: DOS int 21h, функция 02h.
021 ; -----
022 Write_digit_hex PROC NEAR
023     push ax                ; сохраняем AX
024     push dx                ; сохраняем DX
025
026     and al, 0Fh           ; оставляем только младшие 4 бита

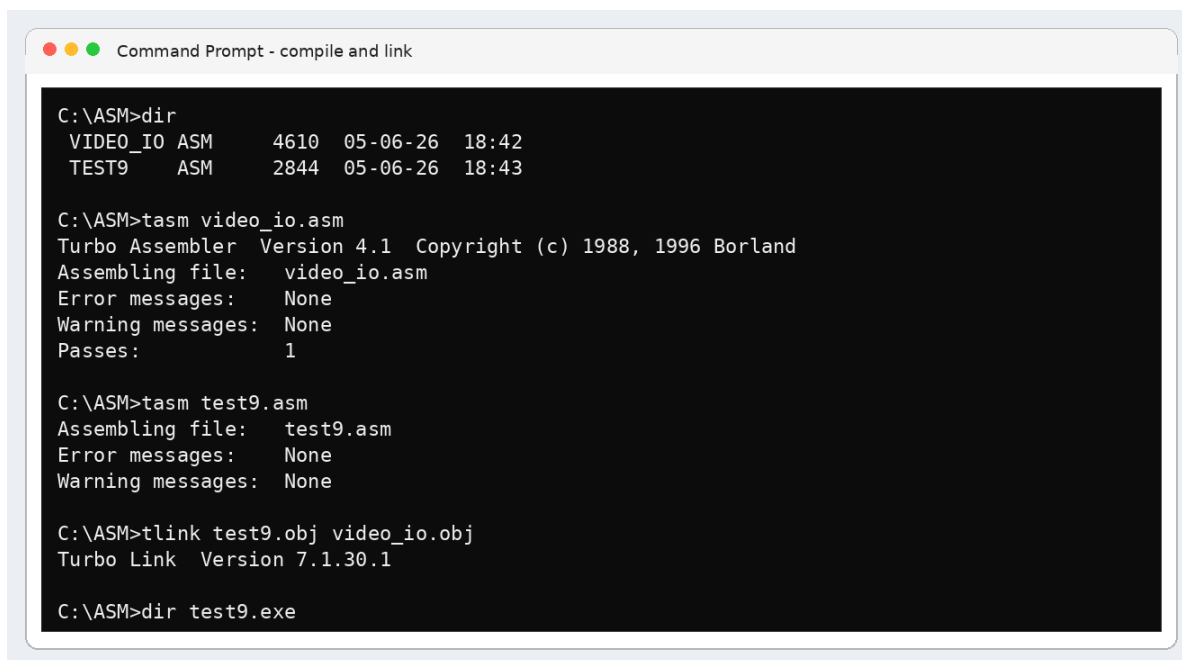
```

Рисунок 1 - файл VIDEO_IO.ASM с процедурами вывода



```
001 ; =====
002 ; Файл: TEST9.ASM
003 ; Назначение: тестирующая программа для процедур из VIDEO_IO.ASM.
004 ; Автор отчёта: Элизбарян Эмин
005 ; =====
006
007 .MODEL SMALL
008 .STACK 100h
009
010 .DATA
011 msg_title db 'LAB 9. WORD HEX OUTPUT TEST',0Dh,0Ah,'$'
012 msg_value db 'Input word = $'
013 msg_alg1 db 'Method 1 = $'
014 msg_alg2 db 'Method 2 = $'
015
016 .CODE
017 EXTRN Write_word_hex_div:NEAR
018 EXTRN Write_word_hex:NEAR
019 EXTRN Write_crlf:NEAR
020
021 start:
022     mov ax, @data
023     mov ds, ax
024
025     lea dx, msg_title
026     call Print_string
```

Рисунок 2 - файл TEST9.ASM с тестирующей программой



```
C:\ASM>dir
VIDEO_IO  ASM      4610  05-06-26  18:42
TEST9     ASM      2844  05-06-26  18:43

C:\ASM>tasm video_io.asm
Turbo Assembler Version 4.1 Copyright (c) 1988, 1996 Borland
Assembling file:  video_io.asm
Error messages:   None
Warning messages: None
Passes:          1

C:\ASM>tasm test9.asm
Assembling file:  test9.asm
Error messages:   None
Warning messages: None

C:\ASM>tlink test9.obj video_io.obj
Turbo Link Version 7.1.30.1

C:\ASM>dir test9.exe
```

Рисунок 3 - компиляция и компоновка исходных файлов

```
Command Prompt - TEST9.EXE

C:\ASM>TEST9.EXE
LAB 9. WORD HEX OUTPUT TEST

Input word = 0000
Method 1   = 0
Method 2   = 0000

Input word = 0009
Method 1   = 9
Method 2   = 0009

Input word = 000A
Method 1   = A
Method 2   = 000A

Input word = 000F
Method 1   = F
Method 2   = 000F

Input word = 0010
Method 1   = 10
Method 2   = 0010

Input word = 00FF
Method 1   = FF
Method 2   = 00FF
```

Рисунок 4 - запуск TEST9.EXE и результаты проверки

```
Command Prompt - DEBUG TEST9.EXE

C:\ASM>debug test9.exe
-u cs:0100
13A2:0100 B8A213      MOV AX,13A2
13A2:0103 8ED8        MOV DS,AX
13A2:0105 BA0000      MOV DX,0000
13A2:0108 E87D00      CALL 0188
13A2:010B E81101      CALL Write_crlf
...
-g
LAB 9. WORD HEX OUTPUT TEST

Input word = 0000
Method 1   = 0
Method 2   = 0000

Input word = FFFF
Method 1   = FFFF
Method 2   = FFFF

Program terminated normally
-q
```

Рисунок 5 - проверка программы через Debug

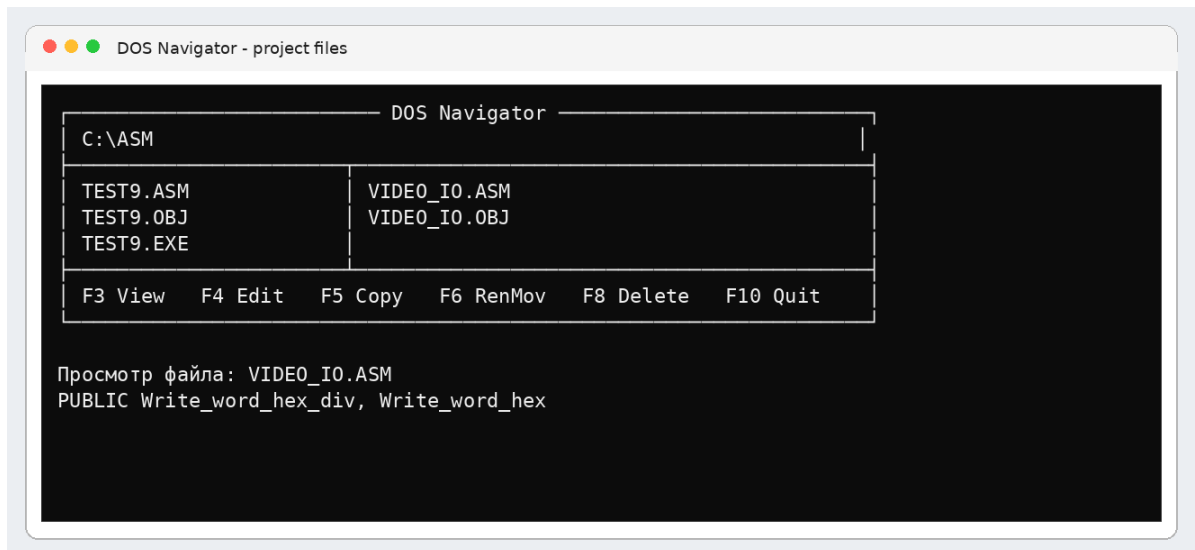


Рисунок 6 - файлы проекта в DOS Navigator

8. Результат работы

В ходе выполнения лабораторной работы были разработаны две процедуры вывода 16-битного числа в шестнадцатеричной системе счисления. Первая процедура формирует число через последовательное деление на 16 и стек остатков. Вторая процедура `Write_word_hex` выводит слово путем двух вызовов процедуры `Write_byte_hex` и обеспечивает отображение результата ровно четырьмя символами.

Тестирование показало корректную работу обеих процедур на граничных значениях 0000h, 000Fh, 0010h, 00FFh и FFFFh. Вторая процедура подходит для дальнейшего использования при выводе четырехзначных адресов памяти.